

Capítulo 5

SIMULACIÓN DE SUCESOS DISCRETOS

Como se ha ilustrado en el Capítulo 1, la simulación esencialmente consiste en experimentar con un modelo computacional de un sistema, aplicándole ciertas entradas, para observar y estudiar sus salidas con el objeto último de predecir el comportamiento del sistema y mejorarlo en su caso. En este capítulo, en su primera parte, exponemos conceptos, como los de sistema, modelo y modelización, de uso necesario en el proceso de simulación, sin entrar en excesivo formalismo. Además, en la descripción en cuatro pasos que hemos hecho en el Capítulo 1 de los experimentos de simulación, describimos el tercero como conversión de las entradas en salidas vía el modelo de simulación. Tal paso dependerá esencialmente de la clase de modelos que consideremos. En la segunda parte del capítulo, estudiamos una clase esencial de modelos de simulación, de sucesos discretos (SSD). Comenzamos discutiendo ideas genéricas sobre SSD y estrategias de SSD y las aplicamos para analizar las medidas de comportamiento de dos sistemas de espera complejos y de un modelo de inventario probabilístico. Para concluir haremos una descripción de los lenguajes y programas de simulación que pueden obtenerse actualmente en el mercado e introduciremos algunas ideas sobre SSD paralela y distribuida. Otros modelos y aplicaciones de la simulación se estudian en los Capítulos 6 y 7.

5.1. MODELIZACIÓN Y SIMULACIÓN

Aunque los orígenes de la simulación científica se sitúan en los trabajos de Student para determinar la distribución de la variable t que lleva su nombre, se suele afirmar que esta disciplina renació, como una técnica numérica, durante la Segunda Guerra Mundial, cuando Von Neumann y Ulam aplicaron los llamados Métodos de Montecarlo (Capítulos 6 y 7) a problemas de difusión de neutrones en el desarrollo de la bomba atómica. Aunque, en sentido amplio, la realización de una simulación no requiere un ordenador, la evolución y el avance de los ordenadores digitales y de los lenguajes de simulación de alto nivel han permitido la aplicación de la modelización y simulación a problemas complejos en numerosas disciplinas, según se describe en capítulos posteriores.

Mediante los términos *modelización* y *simulación* se designa al conjunto de actividades asociadas con la construcción de modelos de sistemas del mundo real y su simulación en un ordenador. Con la definición anterior relacionamos tres elementos: *sistema*, *modelo* y *ordenador*. Por sistema entendemos la parte del mundo real por la que mostramos interés. Consideraremos esa parte de la realidad como un conjunto de elementos o componentes que interaccionan para alcanzar una meta común. Un modelo será una representación simplificada de un sistema con un conjunto de instrucciones válidas para generar datos del comportamiento del sistema.

Los tres elementos anteriores no están aislados entre sí, sino que se conectan a través de las relaciones de *modelización* y *simulación*, véase la Figura 5.1. La modelización relaciona los sistemas reales con los modelos y estudia la *validez* de éstos, es decir, la forma en que un modelo representa un sistema real.

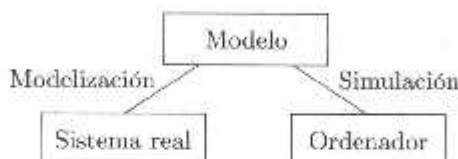


Figura 5.1: Elementos básicos y relaciones de modelización y simulación

La simulación relaciona modelos y ordenadores y describe el proceso de imitación de aspectos importantes del comportamiento del sistema en tiempo real, comprimido o expandido, mediante la construcción y experimentación con el modelo del sistema. La simulación, objeto esencial de este libro, es análoga al proceso de experimentación que llevan a cabo los científicos en un laboratorio con el que

pretenden aumentar la comprensión de alguna teoría para su validación y empleo posterior.

Los sistemas pueden estudiarse mediante experimentación directa, la construcción de un prototipo a escala, o el desarrollo de modelos de relaciones lógicas o cuantitativas que los describan. Razones como coste, riesgo, elevado consumo de tiempo, problemas éticos, o, simplemente, la imposibilidad de llevarlo a cabo, pueden conducirnos a rechazar los dos primeros procedimientos de estudio. Nosotros abordaremos aquí la construcción de modelos cuantitativos. Si pensamos en el diseño de una nueva ciudad o un barrio, resulta impensable construir un prototipo y experimentar, siendo la simulación del modelo el procedimiento adecuado para conocer las implicaciones de ese nuevo diseño. Nuestro propósito será, entonces, el estudio de sistemas reales a través de la modelización cuantitativa para tratar de ayudar en el análisis, diseño, entendimiento, operación, predicción y control de los sistemas sin necesidad de su construcción efectiva y operación real. Los modelos cuantitativos que no son fácilmente tratables con los métodos analíticos o numéricos tradicionales, forman una subclase de modelos, denominada de simulación, que es la que nos interesa aquí.

Como iremos viendo, la modelización y simulación por ordenador está dirigida esencialmente a la búsqueda de soluciones satisfactorias en problemas prácticos. Así, en general, la solución que se propone para un modelo de simulación es una *solución satisfaciente*, es decir, una solución factible que cumple unos estándares mínimos y, por ello, admisible para su implementación, véase la Figura 5.2. Tal solución no coincidirá generalmente con la óptima, pero suele encontrarse lo suficientemente próxima a aquella como para que sea tomada como solución del problema.

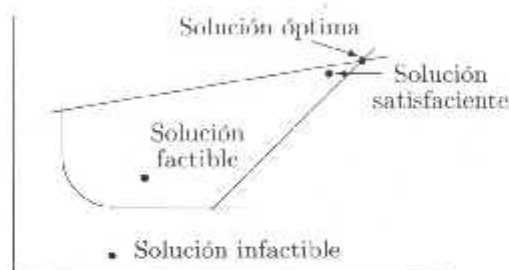


Figura 5.2: Tipos de soluciones

La falta de optimalidad en un sentido estricto puede ser un precio a pagar por el uso de la simulación a cambio de disponer de una solución satisfaciente en un tiempo razonable frente a tener la óptima en un tiempo mucho mayor o, incluso, no llegar a disponer de ella. Además, en problemas complejos no se dispone de resultados que caractericen la solución óptima, por lo que es difícil tener condiciones de parada.

En definitiva, la modelización y simulación con ordenador serán atractivas especialmente cuando el problema bajo estudio sea de resolución difícil, costosa, o imposible con métodos analíticos numéricos clásicos.

5.2. SISTEMAS

5.2.1. Concepto de sistema

El primer elemento que hemos indicado en el proceso de modelización y simulación es el de sistema. Por sistema real entendemos el que, en su forma más elemental, es una *fente de datos* de comportamiento de alguna parte del mundo real por la que mostramos interés. Esa parte de la realidad estará formada por un conjunto de *elementos* o *componentes* o *entidades* que interaccionan para alcanzar un objetivo común. Es más importante la organización de los elementos que actúan de modo conjunto en el sistema que en los elementos mismos.

Los sistemas pueden ser naturales o artificiales, *actuales* o *planificados para un futuro*. Por ejemplo, un centro de cálculo puede considerarse como un sistema artificial que realmente existe, con analistas, programadores, usuarios, ordenadores, impresoras... como componentes. Los elementos poseen ciertas características o *atributos*, *parámetros* y *variables*, que toman valores numéricos o lógicos y, en conjunto, se denominan *variables descriptivas* del sistema. En nuestro ejemplo, los atributos podrían ser el número de ordenadores, las velocidades de procesamiento, las memorias, la capacidad de los analistas, el número de terminales,... Se dan una serie de *relaciones* o *actividades* entre los elementos y, en consecuencia, los elementos interaccionan produciendo cambios en el sistema. Por ejemplo, un centro de cálculo tiene terminales que emplean los analistas. Si no hubiera analistas, no se podrían procesar los programas.

Consideramos relaciones *internas* y *externas* al sistema. Las primeras conectan los elementos dentro del sistema; las segundas conectan los elementos con el mundo exterior. Por ejemplo, una relación interna es la interacción entre analistas y programadores, o entre analistas y terminales. Una relación externa es la llegada de usuarios al centro de cálculo para procesar y depurar sus programas. Las interrelaciones entre entidades y entre entidades y sistema, se expresan a través

de parámetros y de variables. Cualquier proceso que cambie los atributos de una entidad se denomina *actividad*, como, por ejemplo, la llegada o salida de usuarios del centro de cálculo.

Podemos clasificar los sistemas de muchas formas. Por ejemplo, un sistema puede ser *cerrado* o *abierto*. Un sistema cerrado no se relaciona con el medio que le rodea; uno abierto recibe del medio entradas que lo influyen. Si el sistema tiene la capacidad de reaccionar a cambios en su propio estado, entonces el sistema tiene retroalimentación. Además, los sistemas pueden ser *naturales* (una colonia de abejas) o *artificiales* (un automóvil); *dinámicos* (información enviada a través de una serie de canales) o *estáticos* (una lámpara); *estables* (regresan al equilibrio después de una perturbación) o *inestables*; *estocásticos* (incluyen elementos aleatorios) o *determinísticos*; *adaptativos* (responden a cambios en el medio) o *no adaptativos*; *lineales* (todas las relaciones son lineales) o *no lineales*. También, pueden tener variables *independientes* (no manipulables) o *dependientes*; *no controlables* (radiación) o *controlables*; *continuas*, *discretas* o *mixtas*;...

5.2.2. Estados y frontera de un sistema

Con el estudio de sistemas se pretende aprender, diseñar, cambiar, conservar y, si es posible, controlar y mejorar su comportamiento. Algunas de las variables que intervienen y describen el comportamiento de un sistema se denominan *no observables*, ya que no son accesibles para su posible observación y medición. Por el contrario, las variables que pueden medirse se denominan *observables* y podrán ser de *entrada* o de *salida*. Por ejemplo, la sagacidad, la perspicacia y el número medio de líneas que escribe por minuto un programador son tres variables; las dos primeras no son observables ni medibles directamente, pero la tercera sí.

Hemos mencionado que los componentes de un sistema están determinados por un conjunto de variables descriptivas. Las *reglas* que especifican la interacción entre los componentes determinan la forma en que estas variables descriptivas cambian a lo largo del tiempo. Para que un ordenador sea capaz de simular un modelo debe *conocer* estas reglas de interacción. También puede ser que tenga que guardar los valores pasados de las variables descriptivas para poder calcular los valores futuros, ya que las reglas de interacción son función de tales valores. Pero no tienen por qué almacenarse todos los valores pasados de todas las variables descriptivas, pues en muchos modelos resulta posible considerar un subconjunto reducido de variables descriptivas cuyos valores actuales permitan calcular los valores futuros de todas las variables descriptivas. Tal subconjunto mínimo de variables se denomina *variables de estado*. Los valores de las variables de estado en un instante de tiempo t proporcionan el *estado del sistema* en ese instante.

Las variables de estado relacionan el futuro del sistema con el pasado a través del presente. El estado del sistema puede cambiar como resultado de actividades *internas o endógenas*, o bien actividades *externas o exógenas*. En nuestro ejemplo, la caída de un terminal sería una actividad endógena, mientras que la llegada de un usuario sería exógena. Ambos *sucesos* cambiarán el estado del sistema.

Los atributos de los elementos del sistema definen su estado. Si el comportamiento de los elementos puede predecirse con seguridad, estamos ante un *sistema determinístico*. Si no es posible una predicción exacta, entonces nos enfrentamos con un *sistema estocástico*. Las *probabilidades de transición* de un estado a otro en un sistema dado como respuesta a ciertas actividades caracteriza la naturaleza determinista o aleatoria del sistema. Si la probabilidad de que un sistema que está en un estado S_1 pase a un estado S_2 es uno, entonces tenemos un sistema determinístico. Por ejemplo, al enfriar agua a bajas temperaturas se obtiene hielo, o del petróleo se obtiene calor y gas por combustión. En los sistemas estocásticos, la probabilidad de que el sistema cambie de un estado S_1 a un estado $S_2, S_3, \dots$ es menor que la unidad. Por ejemplo, que un alumno supere un examen de una asignatura o que un programa no dé errores en su procesamiento.

Las propiedades estáticas y dinámicas de un sistema describen también el estado del sistema. Un sistema se encuentra *en equilibrio* o *en estado estacionario* si la probabilidad de encontrarse en alguno de los estados no cambia con el tiempo: el sistema puede moverse de un estado a otro, pero las *probabilidades* de su movimiento entre los estados permanecen fijas. En el caso de un sistema evolutivo o dinámico, las probabilidades cambian con el paso del tiempo. En términos matemáticos, la caracterización de los sistemas estáticos se hace generalmente con ecuaciones algebraicas y la de los dinámicos con ecuaciones diferenciales o en diferencias.

Como ya hemos apuntado, un sistema abierto opera en un medio y es posible que los cambios que se produzcan en el medio afecten su comportamiento. Está clara la necesidad de tratar de identificar el *medio* y las *fronteras* del sistema para poder conocer cómo le afecta. El medio del sistema es un subconjunto del mundo exterior formado por aquellos elementos, conceptuales o físicos, que están fuera del sistema y son capaces de influir de manera significativa sobre el comportamiento del mismo, aunque no puedan controlarse desde el sistema. Por ejemplo, un banco es un sistema abierto y la inflación es un elemento de su medio que influye sobre él, pero no puede controlarla.

La frontera del sistema encierra las entidades y relaciones que lo describen, que no tienen por qué ser exclusivas del sistema. Determinadas entidades pueden ser compartidas por diferentes sistemas o subsistemas.

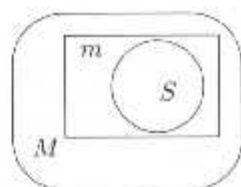


Figura 5.3: Un sistema, su medio y el mundo exterior

La Figura 5.3 muestra un esquema de un sistema y su medio. El sistema S está sumergido en el mundo M (medio global) y los elementos de éste que tienen alguna influencia sobre S constituyen su medio m (medio del sistema). Si no hay interacción entre el sistema y el mundo exterior, entonces m es vacío y tenemos un sistema cerrado. En este caso, el comportamiento del sistema se debe únicamente a las actividades endógenas, es decir, a las interacciones internas del sistema o sus subsistemas. En la práctica, la mayor parte de los sistemas son abiertos e interactúan con su medio cambiando los estados del sistema tanto por actividades endógenas como exógenas.

5.3. MODELOS

5.3.1. Definición de modelo y su clasificación

El primer paso en el estudio de un sistema es la construcción de un modelo. Numerosos autores han discutido la importancia de los modelos y su construcción. por ejemplo, Rosenbluth y Wiener (1945) indican:

Ninguna parte substancial del universo es lo suficientemente simple como para que pueda ser comprendida y controlada sin abstracción. La abstracción consiste en reemplazar la parte del universo bajo consideración por un modelo de estructura similar pero más simple. Los modelos constituyen un elemento central del proceso científico.

Un modelo científico puede definirse como una representación simplificada de un sistema real o un proceso o una teoría, con el que se pretende aumentar su comprensión, hacer predicciones y, posiblemente, ayudar a controlar y mejorar el sistema. Por ello, el propósito de un modelo es capacitar a un individuo para determinar cómo uno o varios cambios en aspectos del sistema modelizado pueden afectarle parcial o globalmente.

El término modelo puede tener diferentes significados para distintas personas y la representación de un modelo puede ser distinta. Una clasificación, de las muchas existentes, considera esencialmente tres clases de modelos:

- *Modelos físicos*. Son representaciones de sistemas físicos y están descritos por variables medibles. Por ejemplo, un sistema mecánico que tiene como variables: fuerza, velocidad, presión... Dentro de esta clasificación se consideran también modelos a escala (avión), de imitación (mapa de relieve), analógicos (reloj de arena) y prototipos (circuito eléctrico).
- *Modelos mentales*. Son modelos heurísticos o intuitivos que sólo existen en nuestras mentes. Son imprecisos, difusos y difíciles de comunicar. A diferencia de los animales, el ser humano es capaz de acumular experiencia que puede servir como un modelo mental. La habilidad para realizar una operación aritmética o el proceso de decisión en una situación no muy compleja son ejemplos de modelos mentales. La introspección de estos modelos conduce frecuentemente a los modelos simbólicos.
- *Modelos simbólicos*. Son aquellos que incluyen operaciones lógicas o matemáticas que pueden utilizarse para formular una solución de un problema. Se construyen más fácilmente y resultan más económicos que los modelos físicos. Se subdividen en modelos *matemáticos* y *no matemáticos*. A su vez, estos últimos pueden ser lingüísticos (descripción verbal), gráficos (dibujo) y esquemáticos (por ejemplo, los diagrama de flujo).

En este libro, nuestro interés se centra en los modelos matemáticos, denominados también cuantitativos, es decir, en modelos descritos por un conjunto de símbolos y relaciones lógico-matemáticas. Los otros tipos de modelos también son importantes, pero no los estudiamos aquí. Ríos (1995) proporciona información adicional.

5.3.2. Metodología de la modelización

La metodología de la modelización se refiere al conjunto de procedimientos sistemáticos basados en conocimiento acumulado *cuyo objetivo es abordar y resolver problemas*.

La *modelización* es el proceso por el cual se establecen relaciones entre las entidades importantes de un sistema que se expresa en términos de metas, criterios de ejecución y restricciones que, en conjunto, constituyen el modelo. Cada modelizador tiene un *modelo básico*, que constituye la visión o imagen particular que

tiene sobre el sistema y, a partir de la cual, se construirá un *modelo simplificado*. Mediante la experimentación con este modelo simplificado, se espera aumentar la comprensión del modelo básico así como del sistema real caracterizado por él.

Un primer paso en la modelización es establecer el problema de forma clara, lógica y no ambigua, delimitando sus fronteras. Puesto que es casi imposible comprender y aislar todas las interrelaciones entre los elementos del sistema, incluiremos sólo un subconjunto de variables e interrelaciones del sistema original. La habilidad para seleccionar el subconjunto más pequeño de variables que describan de forma adecuada el sistema real es una cualidad notable de un modelizador, en el que también desempeñarán un papel importante su experiencia, intuición, imaginación,... Así, un rasgo importante para construir un buen modelo será la simplicidad. Con frecuencia, no existirá correspondencia biunívoca entre las variables del sistema y del modelo y muchos detalles se resumirán dependiendo del nivel de abstracción del mismo. Los modelos con menor número de variables y datos son más fáciles de construir, desarrollar, modificar y comprender, así como más fácilmente tratables, y es muy probable que puedan utilizarse en las situaciones prácticas para las que se han diseñado.

Observemos que la modelización es un *proceso iterativo*. Además, el paso del modelo básico al modelo simplificado se acompaña, generalmente, de una transformación de información cualitativa en cuantitativa. El proceso de modelización, véase la Figura 5.4, es de naturaleza evolutiva, ya que pasa de unas especulaciones iniciales a hipótesis, a modelos generales y, finalmente, a un modelo específico simplificado.

Mencionemos, además, que el ordenador no sólo se utiliza en la simulación, sino también en varias de las etapas del proceso de modelización que incluyen la recogida y depuración de datos, el diseño de experimentos, los contrastes, y la verificación y validación de hipótesis.

5.3.3. Ventajas de la modelización

La cantidad y calidad de la información contenida en los modelos varía de manera notable, aunque todos ellos tienen la característica común de *ayudar a evaluar el resultado de una decisión del mundo real sin llegar a tomar efectivamente la misma*. Puesto que un modelo es una representación de un sistema, permite evaluar decisiones o acciones sin que se lleven a cabo experimentos reales.

Son muchas las ventajas de la utilización de modelos matemáticos. Entre ellas destacamos que permite a los investigadores la organización del conocimiento y las observaciones sobre el sistema, así como las posibles deducciones lógicas que se puedan tener de esta organización; aporta un marco para contrastar el sistema

y sus posibles modificaciones; permite una perspectiva sobre detalles y aspectos relevantes; una mayor y mejor manipulación que con el propio sistema; facilita el análisis; describe un problema de forma más concisa que, por ejemplo, con una descripción verbal; permite controlar mejor las fuentes de variación que lo que permitiría el estudio directo del sistema; y, generalmente, son menos costosos que experimentar con el sistema.

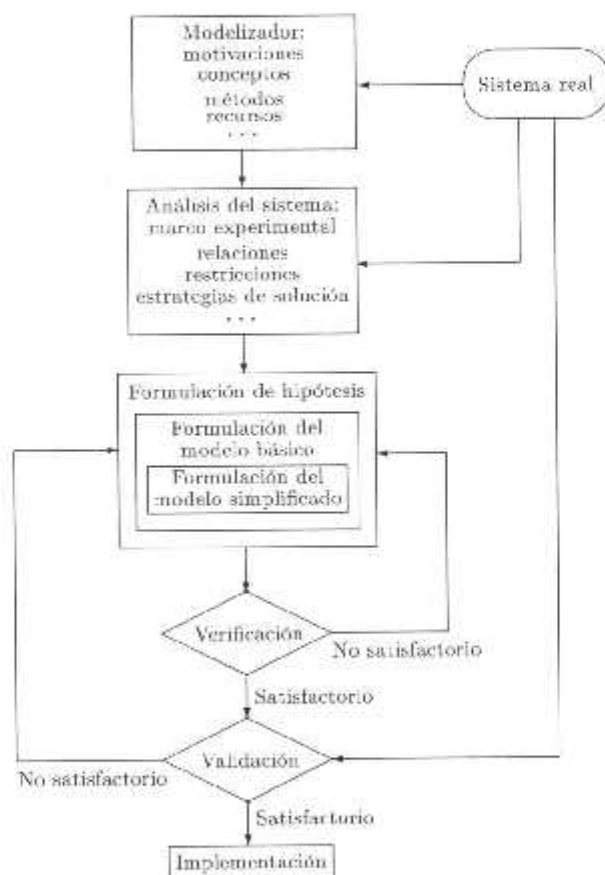


Figura 5.4: Proceso de modelización

Obviamente no todo son ventajas y hay algunas reservas respecto al proceso de modelización. Así, no hay garantía de que a pesar del esfuerzo y tiempo dedicados a la modelización se produzcan resultados útiles y satisfactorios. Por escasez de recursos o falta de ingenio, podrían no alcanzarse las metas deseadas. También, la

tendencia de un investigador a considerar su visión particular del problema como la mejor representación de la realidad, especialmente cuando lleva cierto tiempo en ello, puede conducirle a un modelo de escasa utilidad.

5.3.4. Validez de un modelo

En la construcción de un modelo se busca una *representación válida* de la realidad. Para que un modelo científico sea útil es necesario que incluya elementos de dos atributos contrapuestos: el realismo y la simplicidad. Por un lado, el modelo debe ser una buena aproximación al sistema real e incorporar los aspectos importantes del mismo. Por otro, el modelo no tiene que ser tan complejo como para que resulte difícil de comprender y manejar. Tengamos en cuenta que, al tratarse de una representación formal, un modelo constituye necesariamente una abstracción. La consideración de mayor número de detalles y aspectos confiere al modelo un mejor reflejo de la realidad. Sin embargo, una consecuencia puede ser el aumento de la dificultad en la obtención de una solución de modo que tenga que utilizarse un método numérico de solución. Observemos que no es ni siquiera necesario que la aproximación del modelo al sistema indique la medida de eficacia de las alternativas, sino que lo que se requiere es una correlación alta entre lo que predice el modelo y lo que actualmente ocurre en el sistema real.

La forma de contraste de la validez de un modelo no es única. Existen varios métodos que no sólo no son excluyentes, sino que se complementan y deben tratar de comprobarse todos ellos para tener garantía suficiente. Algunas de estas formas son:

- Reexaminar la formulación del problema para detectar posibles errores y defectos.
- Determinar si todas las expresiones matemáticas son dimensionalmente consistentes.
- Variar los parámetros de entrada y ver que la salida del modelo se comporta de manera admisible.
- Utilizar datos históricos para reconstruir el pasado y determinar lo bien que se habría comportado la solución resultante si se hubiera utilizado. Este contraste retrospectivo, en definitiva, trata de predecir la realidad comparando la eficacia del comportamiento hipotético con lo que ocurrió realmente. Una desventaja de este contraste es que utiliza los mismos datos que posiblemente guiaron la construcción del modelo.

Otro aspecto importante, previo a la validación, es la *verificación* del modelo. Permite asegurar que el modelo se construye de acuerdo con ciertas especificaciones así como eliminar errores en la estructura, en el algoritmo y en la implementación en el ordenador. La comprobación de que el programa es correcto, la convergencia, exactitud, robustez y errores de transcripción serán los rasgos fundamentales de esta fase. Destaquemos aquí la *robustez*, relacionada con el análisis de sensibilidad, que pretende determinar el grado de influencia en la solución del modelo debida a variaciones en los parámetros. Cambiando las condiciones bajo las que se ha construido el modelo, se pueden identificar los parámetros de entrada críticos que afecten de forma notable a la solución. Esto permitiría predecir cambios drásticos en el comportamiento y establecer un procedimiento sistemático de control.

5.3.5. Representación de modelos

Como ayuda a la modelización de un sistema y su simulación, resulta útil dibujar un diagrama que refleje las componentes del sistema y sus relaciones. Aunque es discutible la construcción de tales diagramas para una modelización muy detallada, no cabe duda de que su utilización con la representación de las interacciones más importantes resulta de gran ayuda.

Son varios los tipos de diagramas que se han desarrollado en modelización. En su forma más sencilla, Zeigler *et al.* (2000), la representación de los denominados *diagramas de influencia* (no confundir con los diagramas de influencia utilizados en Análisis de Decisiones) utiliza cajas para indicar las componentes, signos dentro de ellas para las variables descriptivas y parámetros y arcos o flechas para mostrar la influencia o interacción entre pares de componentes. Un ejemplo sencillo correspondiente a un sistema *round robin* o de *compartición de procesador*, se puede observar en la Figura 5.5. El diagrama de influencia de este sistema de tiempo compartido consta de cuatro componentes: un ordenador central (CPU) y tres terminales que utilizan los usuarios (US_i , $i = 1, 2, 3$) para enviar datos. Cada una de estas componentes tiene variables descriptivas y parámetros, las discretas representadas con $\circ$ y las continuas con $\bullet$. La componente CPU contiene la variable $SERVIR_A$ cuyo recorrido es $\{1, 2, 3\}$ y las componentes US_i tienen la variable $COMPLET-s_i$ con recorrido en $[0, 1]$, que indica que el usuario i ha completado una fracción s_i de su tarea. Además, cada componente usuario contiene un parámetro, $PARAM-a_i$, que indica su tasa de compleción de US_i . Finalmente, la existencia de un arco desde las componentes US_i a la componente CPU representan el envío de datos.

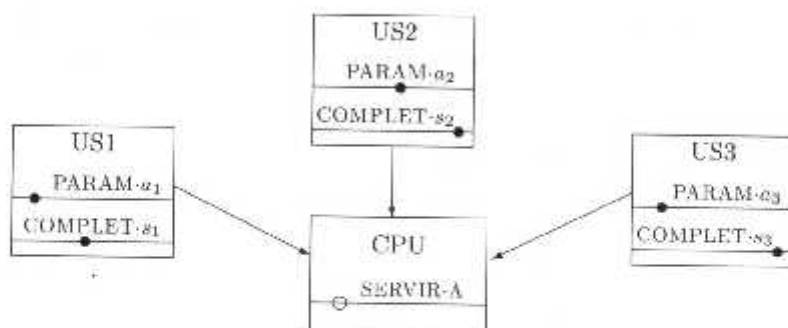


Figura 5.5: Diagrama de influencia de un sistema de round robin

Este tipo de diagramas se ha mejorado y extendido para poder reflejar situaciones más complejas. Así, Forrester (1961, 1971) introduce los *diagramas de nivel de tasa* en los que para cada elemento del sistema se dibuja su propio símbolo con indicación de los niveles mediante rectángulos; mientras que Goodman (1983) y Richardson (1997) consideran los *diagramas de bucles causales*. Una descripción detallada de procedimientos de representación de modelos puede verse en Pidd (2004).

Después de la construcción de un modelo matemático para el problema que se considera, el siguiente paso es obtener una solución a partir del modelo. Llevarémoslo a cabo mediante simulación, a la que dedicaremos, en el caso de sucesos discretos, el resto del capítulo.

5.4. CONCEPTOS BÁSICOS DE LA SIMULACIÓN DE SUCESOS DISCRETOS

Existen distintos tipos de modelos de simulación y entre ellos citamos:

- Los *modelos continuos* se refieren a sistemas cuyo estado cambia de forma continua en el tiempo. Normalmente se representan en forma de sistemas de ecuaciones diferenciales o en diferencias que describen las interacciones entre los distintos elementos del sistema.
- Los *modelos discretos* se refieren a sistemas cuyo comportamiento sólo cambia en instantes determinados. Un ejemplo típico ocurre en las líneas de espera, donde estamos interesados en la estimación de medidas como el tiempo medio de espera o la longitud de la cola. Tales medidas sólo cambian cuando

un cliente entra o sale del sistema. El resto del tiempo, no ocurre nada en el sistema desde el punto de vista de la inferencia estadística.

Los momentos en los que se producen los cambios en el sistema identifican los *sucesos* del modelo. El hecho de que estos sucesos ocurran en instantes discretos da lugar al nombre de *simulación de sucesos discretos*, cuando tratamos de simularlos.

En SSD solemos diferenciar tres tipos de variables:

- La *variable tiempo* t , que indica el tiempo transcurrido de simulación.
- Las *variables contadores*, que contabilizan el número de veces que se han producido los distintos sucesos de interés.
- Las *variables de estado*, subconjunto mínimo de variables que describe el sistema en un instante particular del tiempo, y que permite predecir su comportamiento futuro. Los valores de las variables de estado en un instante t proporcionan el *estado del sistema* en ese instante.

Los sucesos que definen la evolución se generan en distintos instantes de tiempo y el paso del tiempo se controla mediante un *mecanismo de reloj* o *reloj de la simulación*. Básicamente, hay dos tipos de mecanismo de reloj que identifican dos estrategias de simulación, que esbozamos a continuación.

En la *simulación síncrona* u *orientada a intervalos*, el incremento del reloj se fija en una cantidad Δt , al término de la cual se actualizan las estadísticas y se cambia el estado del sistema. El proceso se repite hasta que se cumpla una condición de finalización, momento en el cual se imprimen los resultados y se concluye la simulación, como se expresa en la Figura 5.6. Este tipo de control del mecanismo del reloj de simulación tiene como inconveniente la selección del incremento de tiempo Δt . Si tomamos un incremento demasiado grande, podría producirse más de un suceso durante ese intervalo de tiempo y el orden en que se ha producido no se habría registrado ya que se asume que han tenido lugar al final del mismo. Si, por el contrario, lo tomamos demasiado pequeño, puede que en la mayoría de intervalos de tiempo no ocurra ningún suceso, lo que llevará asociado una ineficiencia computacional. Por ello, y a pesar de su sencilla implementación, en la SSD utilizaremos la segunda alternativa para el control del mecanismo del reloj de simulación, la *simulación asíncrona*. Nótese, sin embargo, que la simulación síncrona es una estrategia valiosa para sistemas en tiempo continuo que, aunque no se tratarán en el presente libro, se estudian en, por ejemplo, Neelamkavil (1987). Como ejemplo de herramienta que desarrolla un mecanismo

de simulación síncrono podemos destacar el paquete de software *SPEEDES*, véase Bailey *et al.* (2001).

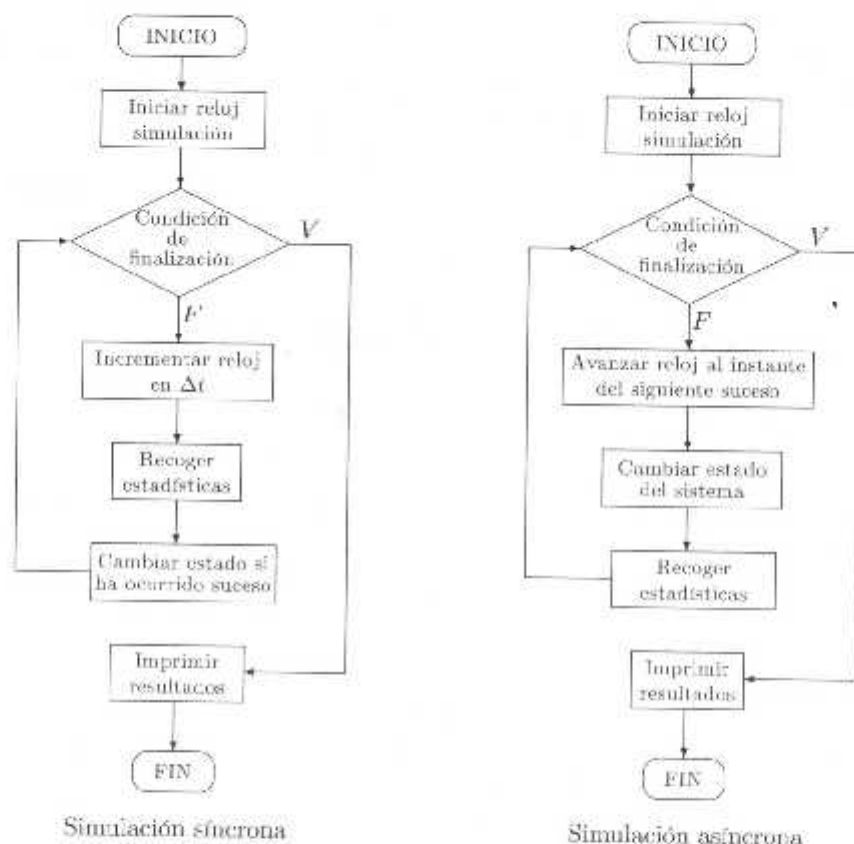


Figura 5.6: Tipos de mecanismos de reloj

En la *simulación asíncrona u orientada a sucesos* el reloj de simulación avanza hasta el instante en el que ocurre un nuevo suceso, momento en el que se actualiza el estado del sistema y se recogen estadísticas. El proceso se repite hasta que se cumpla una condición de finalización, imprimiéndose después los resultados y concluyendo así la simulación, véase la Figura 5.6. Esta estrategia se utiliza más habitualmente en los lenguajes de SSD, por lo que la describiremos en mayor profundidad.

Al depender la evolución de la simulación del sistema de los instantes en los que suceden los sucesos, debemos mantener una lista de éstos y de los instantes

en que tendrán lugar, de forma que el mecanismo de reloj avanzará al instante en el que se produzca el primero de los sucesos de la lista.

Es importante señalar que dado un tipo de suceso determinado, el tiempo que transcurre entre ocurrencias consecutivas del mismo puede ser determinístico (por ejemplo, cada 30 minutos) o aleatorio, en cuyo caso se deberán generar valores a partir de la distribución de probabilidad que describa dicho comportamiento aleatorio.

En las próximas dos secciones se presentarán varios ejemplos representativos de aplicación de la SSD, los modelos de colas y los de inventario probabilísticos. Todos ellos servirán para ilustrar los conceptos e ideas descritos en esta sección.

5.5. SSD DE SISTEMAS DE ESPERA

En numerosas situaciones de nuestra vida diaria esperamos en una cola o línea de espera, como para comprar el billete del metro o la entrada de cine, para pagar en el supermercado, para obtener una mesa en un restaurante o para ser operado o atendido en un hospital. También en los sistemas informáticos son frecuentes los fenómenos de espera. Así, puede haber una cola de personas esperando para usar un terminal, colas de solicitudes de entrada/salida (E/S), mensajes o paquetes de datos o programas informáticos que esperan para ser procesados por un sistema central o llamadas telefónicas esperando una línea desocupada para completar la conexión.

La espera se produce porque hay más demanda de servicio que el disponible. Sin embargo, ampliar esta capacidad de servicio no siempre es la solución adecuada. Para llegar a una solución, el analista del sistema necesita conocer las respuestas a preguntas como: ¿cuánto tiempo debe esperar un cliente?, ¿cuántos clientes se acumularán en la cola? o ¿cuántos clientes llegan por unidad de tiempo? A partir de ellas podrá considerar varios sistemas alternativos y tratar de evaluar su funcionamiento.

La *teoría de colas*, véanse Kleinrock (1975), Ríos Insua *et al.* (2004) y la Sección A.6.4 del Apéndice, proporciona al planificador diversos modelos que dan respuesta a estas cuestiones y, en particular, ha demostrado ser una de las áreas más fructíferas de la Teoría de la Probabilidad aplicada a la Informática. Sin embargo, cuando no existan procedimientos analíticos para analizar el comportamiento del sistema, porque se trate de una red de colas muy grande con disciplinas de espera complejas, tiempos de llegada o de servicio arbitrarios, capacidad finita de ciertos nodos y otros con comportamiento no estable, la SSD nos permitirá predecir las medidas de interés del sistema.

Realizaremos una SSD de un modelo de colas con un único servidor $G/G/1$. En él tanto los tiempos entre llegadas de clientes al sistema como los tiempos de servicio siguen una distribución general, que no tiene por qué ser exponencial y, a continuación, un modelo de red de colas con tres nodos, algunos con capacidad finita y con distintas distribuciones para el tiempo de servicio, algunas de las cuales dependerán del número de clientes en el nodo correspondiente.

5.5.1. Modelo de colas $G/G/1$

Simulemos el comportamiento de un *modelo de colas* con un único servidor, capacidad infinita y distribuciones generales para los tiempos entre llegadas de clientes al sistema y los tiempos de servicio. Supondremos una *disciplina de cola* FIFO y que los tiempos de servicio de los clientes son independientes entre sí e independientes de las llegadas entre clientes. La Figura 5.7 representa dicho sistema. En él, si al llegar un cliente lo encuentra vacío entonces pasa directamente a ser servido. Por el contrario, si ya hay clientes en el sistema, pasa a formar parte de la cola, en la que esperará a que le llegue su turno. Una vez que reciba el servicio demandado, abandonará el sistema.

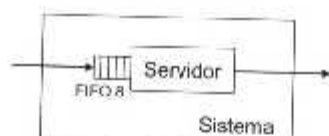


Figura 5.7: Modelo de cola $G/G/1$

El sistema permite la entrada de clientes en el mismo hasta el instante T , momento en el cual no se permiten más entradas, pero sí se sigue dando servicio a los clientes que están en el sistema hasta que todos ellos lo abandonan. Estamos interesados en conocer el tiempo transcurrido desde el instante T hasta que el último cliente abandona el sistema y el tiempo que pasan los clientes en el sistema y en la cola.

En el modelo de colas considerado, los posibles sucesos que se pueden producir son dos, la llegada de un cliente al sistema y la conclusión de servicio de un cliente, lo que implica su salida. Está claro que ambos modificarán el estado del sistema. En el primer caso, se aumentará el número de clientes en el mismo (pudiendo pasar el cliente a la cola o directamente a ser servido), mientras que en el segundo disminuirá este valor. Utilizaremos una estructura de datos con dos elementos en los que guardaremos los siguientes tiempos en los que se producirán los dos tipos de sucesos, $TSuc.\{t_{LL}, t_S\}$. Esta estructura nos permitirá saber en todo momento

si hay algún suceso pendiente y, si es así, en qué instante se producirá, t_{LL} para las llegadas y t_S para la conclusión de servicio. Otras variables que serán también necesarias para realizar la SSD del sistema propuesto son:

- $t \equiv$ tiempo transcurrido de simulación
- $n \equiv n^o$ de clientes en el sistema en el instante t (variable de estado). Obsérvese que el número de clientes en la cola será $n - 1$, salvo cuando el sistema esté vacío.
- $N_{LL}, N_S \equiv n^o$ de llegadas y de salidas hasta el instante t (variables contador), respectivamente.
- $LL(i) \equiv$ instante en el que llega el cliente i -ésimo al sistema.
- $S(i) \equiv$ instante en el que sale del sistema el cliente i -ésimo.
- $Serv(i) \equiv$ tiempo de servicio recibido por el cliente i -ésimo.

Utilizaremos las siguientes tres últimas variables para informar sobre algunos de los tiempos solicitados, en el sistema y en la cola, a través de sus medias.

- $t_med_sistema =$ tiempo medio que pasan los clientes en el sistema (variable de salida).
- $t_med_cola =$ tiempo medio que pasan los clientes en la cola (variable de salida).
- $T_p \equiv$ tiempo transcurrido desde T hasta que el último cliente abandona el sistema (variable de salida).

La estructura del programa principal de simulación de este ejemplo se repite en la de los restantes ejemplos que se analizan en este capítulo, así como en la mayoría de las SSD. En primer lugar, se inicializará el valor de las variables consideradas, señalando además que no hay pendiente ningún suceso (asignando un valor M muy grande en la estructura $TSuc$). Posteriormente, generaremos el instante de tiempo en el que se producirá el primer suceso y se llamará a la rutina que trate su ocurrencia, entrándose, a continuación, en un bucle del que no saldremos mientras haya algún suceso pendiente. Dentro del bucle comprobaremos cuál de todos los posibles sucesos será el primero que se producirá y llamaremos a la rutina correspondiente para tratarlo. Finalmente, una vez que no queden sucesos pendientes y, por lo tanto, haya terminado la simulación, calcularemos las medidas

de comportamiento del sistema en las que estábamos interesados a partir de las variables contador.

Para el problema que nos ocupa, una vez inicializadas las variables, generamos el instante de tiempo en el que se produce la primera llegada de un cliente al sistema, ya que éste es el primer suceso que se producirá (no se puede dar servicio a un cliente si no hay ninguno en el sistema). Si la ocurrencia de dicho suceso cae fuera del horizonte de planificación T , hemos terminado la simulación. En caso contrario, se llama a la rutina que trata la ocurrencia de dicho suceso (Llegada). Dentro del bucle vamos comprobando cuál de los dos posibles sucesos es el primero que se produce y llamamos a la rutina correspondiente para tratarlo. Finalmente, una vez concluida la simulación, calculamos los tiempos medios que pasan los clientes en la cola y en el sistema y T_p .

Hacer $t = t_{suc} = N_{LL} = N_S = n = 0$

Hacer $TSuc.t_{LL} = TSuc.t_S = M$

Hacer $LL(i) = S(i) = Serv(i) = 0 \quad \forall i$

Generar $X \sim F$

Si $X > T$,

Hacer $T_p = t_med_sistema - t_med_cola - 0$

Salir

en caso contrario,

Ejecutar **Llegada**(X)

Mientras $TSuc.t_{LL} \neq M$ ó $TSuc.t_S \neq M$,

Si $TSuc.t_{LL} < TSuc.t_S$,

Hacer $t_{suc} = TSuc.t_{LL}$, $TSuc.t_{LL} = M$

Ejecutar **Llegada**(t_{suc})

Si $TSuc.t_S < TSuc.t_{LL}$,

Hacer $t_{suc} = TSuc.t_S$, $TSuc.t_S = M$

Ejecutar **Servidor**(t_{suc})

Hacer $T_p = \max(0, t - T)$

Hacer $acumulol = acumulol2 = 0$

Hacer $ind = 0$

Mientras $ind < N_{LL}$,

Hacer $acumulol = acumulol + S(ind) - LL(ind)$

Hacer $acumulol2 = acumulol2 + S(ind) - LL(ind) - Serv(ind)$

Hacer $ind = ind + 1$

Hacer $t_med_sistema = acumulol / N_{LL}$

Hacer $t_med_cola = acumulol2 / N_{LL}$

Salir

En la rutina que trata la llegada de clientes al sistema, en primer lugar, actualizamos el tiempo de simulación al instante actual en el que se ha producido la llegada, incrementamos el número de clientes en el sistema y el número de llegadas al mismo y registramos el instante en el que llega el cliente. A continuación, generamos el tiempo que transcurre hasta la llegada del siguiente cliente y, si ese instante está dentro del horizonte de planificación, lo guardamos en la lista de sucesos. Finalmente, si el número de clientes en el sistema es 1, es decir, el cliente que acaba de llegar ha encontrado el sistema vacío, generamos su tiempo de servicio, lo guardamos en la lista de sucesos y registramos el tiempo de servicio correspondiente a ese cliente.

Llegada (*int* t_{suc})

Hacer $t = t_{suc}$, $n = n + 1$, $N_{LL} = N_{LL} + 1$, $LL(N_{LL}) = t$

Generar $X \sim F$

Si $t + X \leq T$, hacer $TSuc.t_{LL} = t + X$

Si $n = 1$,

 Generar $Y \sim G$

 Hacer $TSuc.t_S = t + Y$, $Serv(N_S + 1) = Y$

En la rutina que trata la conclusión del servicio de un cliente, en primer lugar, actualizamos el tiempo de simulación al instante en el que se produce el suceso, disminuimos el número de clientes en el sistema, incrementamos el número de salidas del mismo y registramos el instante en el que sale el cliente. A continuación, si aún hay clientes en el sistema, generamos el tiempo que transcurre hasta la conclusión del servicio del siguiente cliente, lo guardamos en la lista de sucesos y registramos el tiempo de servicio correspondiente a ese cliente.

Servidor (*int* t_{suc})

Hacer $t = t_{suc}$, $n = n - 1$, $N_S = N_S + 1$, $S(N_S) = t$

Si $n > 0$,

 Generar $Y \sim G$

 Hacer $TSuc.t_S = t + Y$, $Serv(N_S) = Y$

Evidentemente, si el programa principal se ejecuta una única vez, se obtendrá una estimación bastante pobre de las cantidades demandadas. Por ello, será necesario replicar la ejecución del mismo obteniendo distintas estimaciones, las cuales serán analizadas mediante las técnicas introducidas en el Capítulo 8, de análisis de los resultados.

5.5.2. Red de colas

Analizamos ahora el funcionamiento de un Servicio de Inspección Técnica de Vehículos (ITV). Los clientes llegan en sus turismos a las instalaciones de la ITV según un proceso de Poisson de parámetro λ , siendo atendidos inicialmente en una ventanilla en la que entrega la documentación del vehículo y pagan las tasas correspondientes. El tiempo empleado para ello se distribuye según una normal con media μ_1 y desviación típica σ_1 (debiendo truncarse para no tomar valores negativos). A continuación, el cliente dirige su vehículo a las instalaciones de pruebas. Las pruebas que se realizan al vehículo dependen de la antigüedad del mismo, teniendo que pasar unas pruebas adicionales en una nave auxiliar si la antigüedad es superior a los 10 años, lo cual ocurre en el 40% de los casos. El tiempo que se tarda en realizar las pruebas adicionales sigue una distribución exponencial de parámetro μ_2 . El tiempo dedicado a la realización de las pruebas estándar, es decir, aquéllas que tienen que pasar todos los vehículos en la nave principal, se distribuye según una normal, también truncada, cuyos parámetros dependen del número de vehículos que están esperando para realizar las pruebas, ya que se ha comprobado empíricamente que los trabajadores se suelen dar más prisa cuando hay muchos clientes esperando. En concreto, la media es μ_{31} y la desviación típica σ_{31} cuando el número de vehículos esperando es menor que 5, y μ_{32} y σ_{32} cuando es mayor o igual a esa cantidad ($\mu_{31} > \mu_{32}$). Las pruebas adicionales, en caso de que sean necesarias, se realizarán antes que las pruebas estándar.

El sistema descrito se puede representar gráficamente como se muestra en la Figura 5.8. El primer nodo se corresponde con la ventanilla donde se entregan la documentación y tasas, el segundo nodo es la nave auxiliar donde se realizan las pruebas adicionales a los vehículos antiguos y en el tercero, por el que pasan todos los vehículos, se realizan las pruebas estándar y los clientes esperan a que se les devuelva la documentación y el resultado de la inspección. Obsérvese que en los tres nodos existe la posibilidad de que el cliente tenga que hacer cola, asumiéndose un tamaño de la misma infinito y una *disciplina FIFO*.

Las instalaciones de la ITV permiten la entrada de clientes en las mismas hasta el instante T , momento en el cual no se permiten más entradas, pero sí se sigue dando servicio a los vehículos que están en el sistema hasta que todos ellos lo abandonan. Pretendemos determinar el tiempo medio que lleva a los clientes realizar la inspección técnica de sus turismos, el número medio de clientes en cada nodo y el tiempo transcurrido desde el instante T hasta que el último vehículo abandona el sistema.

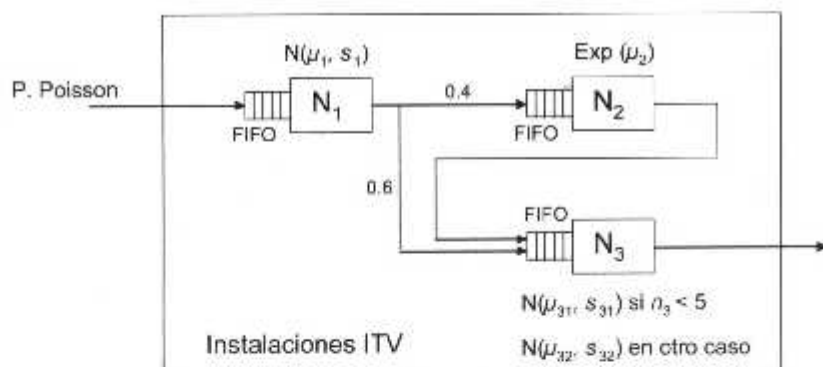


Figura 5.8: Red de colas de las instalaciones de la ITV

En el *modelo de red de colas* considerado, para el cual no existen procedimientos analíticos de solución, los posibles sucesos que se pueden producir son cuatro: la llegada de un cliente a las instalaciones y la conclusión del servicio en cualquiera de los tres nodos, es decir, que se termine de atender a un cliente en ventanilla, que se terminen las pruebas adicionales de un vehículo y que un cliente recoja su documentación y el resultado de la inspección tras realizar las pruebas estándar, abandonando de esta forma el sistema. Utilizaremos una estructura de datos, como en el ejemplo anterior, con cuatro elementos, $TSuc.\{t_{LL1}, t_{S1}, t_{S2}, t_{S3}\}$, que nos permitirá saber en todo momento si hay algún suceso pendiente y, si es así, en qué instante se producirá.

Otras variables que serán también necesarias para realizar la SSD del sistema propuesto son:

- $t \equiv$ tiempo transcurrido de simulación.
- $n_1, n_2, n_3 \equiv$ número de clientes en el nodo 1, 2 y 3, respectivamente (incluidos los que están esperando en cola, que son las variables de estado). Obsérvese que el número de clientes en el sistema es la suma de estas tres variables.
- $N_{LL1}, N_{LL2}, N_{LL3} \equiv$ número de llegadas al primer, segundo y tercer nodos hasta el instante t (variables contador), respectivamente.
- $N_{S1}, N_{S2}, N_{S3} \equiv$ número de salidas del primer, segundo y tercer nodos hasta el instante t (variables contador), respectivamente.
- $LL_j(i), S_j(i) \equiv$ instante en el que llega/sale el cliente i -ésimo al/del servidor j -ésimo, $j = 1, 2, 3$, respectivamente.

Utilizaremos estas seis últimas variables para informar sobre los tiempos solicitados, a través de sus medias.

- $n_med_n_j \equiv$ número medio de clientes en el servidor j -ésimo, $j = 1, 2, 3$ (variables de salida).
- $t_med_sistema \equiv$ tiempo medio que pasan los clientes en el sistema (variable de salida).
- $T_p \equiv$ tiempo transcurrido desde T hasta que el último cliente abandona el sistema (variable de salida).

El programa principal de simulación seguirá la misma estructura general presentada en el ejemplo anterior. Una vez inicializadas las variables, generamos el instante de tiempo en el que se produce la primera llegada de un cliente al sistema, ya que éste es el primer suceso que se producirá (no se puede dar servicio a un cliente en ningún nodo si no hay ninguno en el sistema). Si la ocurrencia de dicho suceso cae fuera del horizonte de planificación T hemos terminado la simulación. En caso contrario, se llama a la rutina que trata la ocurrencia de dicho suceso (Llegada_cliente). Dentro del bucle vamos comprobando cuál de los cuatro posibles sucesos es el primero que se produce y llamamos a la rutina correspondiente para tratarlo. Finalmente, una vez concluida la simulación, calculamos el tiempo medio que pasan los clientes en el sistema, el número medio de clientes en cada nodo y T_p .

Es importante señalar que el orden en el que llegan los clientes a las instalaciones de la ITV no tiene por qué coincidir con el que lo abandonan, ya que un cliente puede adelantar a otros si no necesita realizar las pruebas adicionales y éstos sí. Por lo tanto, el cálculo del tiempo medio de los clientes en el sistema no puede hacerse directamente a partir de las variables contador $S_3(i)$ y $LL_1(i)$, de forma análoga a como se hizo en el modelo $G/G/1$. En esta ocasión, el tiempo medio en el sistema se calculará sumando los tiempos medios de los clientes en el primer y tercer nodo más el tiempo medio en el segundo por la probabilidad de que los clientes pasen por él (0.4). Por otro lado, para obtener el número medio de clientes en cada uno de los nodos, en el tratamiento de los distintos sucesos iremos acumulando en las variables $n_med_n_j$ el producto del número de clientes en el nodo que corresponda por el tiempo que se ha mantenido ese número de clientes hasta la ocurrencia del suceso que se está tratando. De esta forma, al final de la simulación simplemente tendremos que dividir el valor acumulado en dichas variables por el tiempo total de simulación t .

```

Hacer  $t = t_{suc} = T_p = 0$ 
Hacer  $TSuc.t_{LL1} = TSuc.t_{S1} = TSuc.t_{S2} = TSuc.t_{S3} = M$ 
Hacer  $N_{LL1} = N_{LL2} = N_{LL3} = N_{S1} = N_{S2} = N_{S3} = n_1 = n_2 = n_3 = 0$ 
Hacer  $LL_1(i) = LL_2(i) = LL_3(i) = S_1(i) = S_2(i) = S_3(i) = 0, \forall i$ 
Hacer  $n\_med\_n_1 = n\_med\_n_2 = n\_med\_n_3 = 0$ 
Generar  $X \sim \text{Exp}(\lambda)$ 
Si  $X > T$ ,
    Hacer  $T_p = t\_medio\_sistema = 0$ 
    Hacer  $n\_med\_n_1 = n\_med\_n_2 = n\_med\_n_3 = 0$ 
    Salir
en caso contrario,
    Ejecutar Llegada_cliente( $X$ )
    Mientras  $(TSuc.t_{LL1} \text{ ó } TSuc.t_{S1} \text{ ó } TSuc.t_{S2} \text{ ó } TSuc.t_{S3}) \neq M$ ,
        Si  $\min(TSuc) = TSuc.t_{LL1}$ ,
            Hacer  $t_{suc} = TSuc.t_{LL1}, TSuc.t_{LL1} = M$ 
            Ejecutar Llegada_cliente( $t_{suc}$ )
        Si  $\min(TSuc) = TSuc.t_{S1}$ ,
            Hacer  $t_{suc} = TSuc.t_{S1}, TSuc.t_{S1} = M$ 
            Ejecutar Servicio_nodo1( $t_{suc}$ )
        Si  $\min(TSuc) = TSuc.t_{S2}$ ,
            Hacer  $t_{suc} = TSuc.t_{S2}, TSuc.t_{S2} = M$ 
            Ejecutar Servicio_nodo2( $t_{suc}$ )
        Si  $\min(TSuc) = TSuc.t_{S3}$ ,
            Hacer  $t_{suc} = TSuc.t_{S3}, TSuc.t_{S3} = M$ 
            Ejecutar Servicio_nodo3( $t_{suc}$ )
    Hacer  $T_p = \max(0, t - T)$ 
    Hacer  $acumulo1 = acumulo2 = acumulo3 = 0$ 
    Hacer  $ind = 0$ , Mientras  $ind < N_{LL1}$ ,
        Hacer  $acumulo1 = acumulo1 + S_1(ind) - LL_1(ind), ind = ind + 1$ 
    Hacer  $ind = 0$ , Mientras  $ind < N_{LL2}$ ,
        Hacer  $acumulo2 = acumulo2 + S_2(ind) - LL_2(ind), ind = ind + 1$ 
    Hacer  $ind = 0$ , Mientras  $ind < N_{LL3}$ ,
        Hacer  $acumulo3 = acumulo3 + S_3(ind) - LL_3(ind), ind = ind + 1$ 
    Hacer  $t\_med\_sistema = (acumulo1/N_{LL1}) + (0.4 \times acumulo2/N_{LL2}) +$ 
         $+(acumulo3/N_{LL3})$ 
    Hacer  $n\_med\_n_1 = n\_med\_n_1/t$ 

```

```

Hacer  $n\_med\_n2 = n\_med\_n2/t$ 
Hacer  $n\_med\_n3 = n\_med\_n3/t$ 
Salir

```

En la rutina que trata la llegada de clientes a las instalaciones, en primer lugar, actualizamos la variable que acumula el producto del número de clientes en el primer nodo por el tiempo que se ha mantenido ese número de clientes, aumentamos el número de clientes en el primer nodo y el número de llegadas al mismo, y registramos el instante en el que llega el cliente. A continuación, actualizamos el tiempo de simulación al instante en que se produce este suceso, generamos el tiempo que transcurre hasta la llegada del siguiente cliente y si ese instante está dentro del horizonte de planificación lo guardamos en la lista de sucesos. Finalmente, si sólo hay un cliente en ventanilla, es decir, el cliente que acaba de llegar la encuentra vacía, generamos su tiempo de servicio y lo guardamos en la lista de sucesos.

Llegada_cliente (t_{suc})

```

Hacer  $n\_med\_n1 = n\_med\_n1 + n1 \times (t_{suc} - t)$ 
Hacer  $n1 = n1 + 1$ ,  $N_{LL1} = N_{LL1} + 1$ ,  $LL1(N_{LL1}) = t_{suc}$ ,  $t = t_{suc}$ 
Generar  $Y \sim Exp(\lambda)$ 
Si  $t + Y < T$ , hacer  $TSuc.t_{LL1} = t + Y$ 
Si  $n1 = 1$ , generar  $Y \sim \mathcal{N}(\mu_1, \sigma_1)$ , hacer  $TSuc.t_{S1} = t + Y$ 
Salir

```

En la rutina que trata la conclusión del servicio de un cliente en la ventanilla, en primer lugar, actualizamos la variable que acumula el producto del número medio de clientes en el primer nodo por el tiempo que se ha mantenido ese número de clientes, disminuimos el número de clientes en él, aumentamos el número de salidas y registramos el instante en el que sale el cliente. A continuación, el cliente irá al nodo 2 con una probabilidad 0.4 o al nodo 3 con la probabilidad complementaria: generamos un valor de la distribución $\mathcal{U}(0, 1)$; si dicho valor es menor o igual que 0.4, el cliente sale del primer nodo y entra en el segundo para realizar las pruebas adicionales, luego actualizamos la variable que acumula el número medio de clientes en el segundo nodo, aumentamos el número de clientes en él y el número de entradas y registramos el instante en el que entra el cliente. Además, si el nuevo cliente en el segundo nodo lo encuentra vacío, generamos su tiempo de servicio. Si el valor generado de la uniforme es mayor que 0.4, el cliente irá al tercer nodo para pasar las pruebas estándar, en cuyo caso actualizamos la variable que acumula el número medio de clientes en el tercer nodo, aumentamos el número de clientes en él y el número de entradas y registramos el instante en

el que llega el cliente. De nuevo, si este cliente encuentra vacío el tercer nodo, generamos su tiempo de servicio. Finalmente, actualizamos el tiempo de simulación al instante en que se produce este suceso, comprobamos si quedan clientes esperando a ser atendidos en ventanilla y, si es así, generamos el tiempo en que concluirá el servicio del próximo cliente y lo registramos en la lista de sucesos.

Servicio_nodo1 (t_{suc})

Hacer $n_med_n1 = n_med_n1 + n1 \times (t_{suc} - t)$

Hacer $n1 = n1 - 1$, $N_{S1} = N_{S1} + 1$, $S1(N_{S1}) = t_{suc}$

Generar $U \sim \mathcal{U}(0, 1)$

Si $U \leq 0.4$,

Hacer $n_med_n2 = n_med_n2 + n2 \times (t_{suc} - t)$

Hacer $n2 = n2 + 1$, $N_{LL2} = N_{LL2} + 1$, $LL2(N_{LL2}) = t_{suc}$

Si $n2 = 1$, generar $Z \sim \text{Exp}(\mu_2)$, hacer $TSuc.t_{S2} = t_{suc} + Z$

en caso contrario,

Hacer $n_med_n3 = n_med_n3 + n3 \times (t_{suc} - t)$

Hacer $n3 = n3 + 1$, $N_{LL3} = N_{LL3} + 1$, $LL3(N_{LL3}) = t_{suc}$

Si $n3 = 1$, generar $W \sim \mathcal{N}(\mu_{31}, \sigma_{31})$, hacer $TSuc.t_{S3} = t_{suc} + W$

Hacer $t = t_{suc}$

Si $n1 > 0$, generar $S \sim \mathcal{N}(\mu_1, \sigma_1)$, hacer $TSuc.t_{S1} = t + S$

Salir

En la rutina que trata la conclusión del servicio de un cliente en el segundo nodo, actualizamos la variable que acumula el producto del número medio de clientes en el segundo nodo por el tiempo que se ha mantenido ese número de clientes, disminuimos el número de clientes, incrementamos el número de salidas y registramos el instante en el que sale el cliente. Además, tenemos que generar el tiempo de servicio del siguiente vehículo si es que hay alguno en el nodo esperando. A continuación, teniendo en cuenta que los clientes que terminan su servicio en el segundo nodo pasan al tercero para realizar las pruebas estándar, actualizamos la variable que acumula el número medio de clientes en el tercer nodo, aumentamos el número de clientes en él y el número de entradas y registramos el instante en el que entra el cliente. Si el cliente que llega al tercer nodo lo encuentra vacío, entonces debemos generar su tiempo de servicio y registrarlo en la lista de sucesos. Finalmente, actualizamos el tiempo de simulación al instante en que se produce este suceso.

Servicio_nodo2 (t_{suc})

Hacer $n_med_n2 = n_med_n2 + n2 \times (t_{suc} - t)$
 Hacer $n2 = n2 - 1$, $N_{S2} = N_{S2} + 1$, $S_2(N_{S2}) = t_{suc}$
 Si $n2 > 0$, generar $Y \sim \text{Exp}(\mu_2)$, $TSuc.t_{S2} = t_{suc} - Y$
 Hacer $n_med_n3 = n_med_n3 + n3 \times (t_{suc} - t)$
 Hacer $n3 = n3 + 1$, $N_{LL3} = N_{LL3} + 1$, $LL_3(N_{LL3}) = t_{suc}$
 Si $n3 = 1$, generar $W \sim \mathcal{N}(\mu_{31}, \sigma_{31})$, hacer $TSuc.t_{S3} = t_{suc} + W$
 Hacer $t = t_{suc}$;
 Salir

En la rutina que trata la conclusión del servicio en el tercer nodo, actualizamos la variable que acumula el producto del número medio de clientes en el tercer nodo por el tiempo que se ha mantenido ese número de clientes, disminuimos el número de clientes en él, aumentamos el número de salidas y registramos el instante en el que sale el cliente. Además, tenemos que generar el tiempo de servicio del siguiente cliente si es que hay alguno esperando para realizar las pruebas estándar. Recordemos que la distribución del tiempo de servicio en este nodo depende del número de clientes en el mismo. Por lo tanto, debemos comprobar si el número de clientes es mayor que cinco y utilizar la distribución normal truncada con los parámetros que corresponda. Finalmente, actualizamos el tiempo de simulación al instante en que se produce este suceso.

Servicio_nodo3 (t_{suc})

Hacer $n_med_n3 = n_med_n3 + n3 \times (t_{suc} - t)$
 Hacer $n3 = n3 - 1$, $N_{S3} = N_{S3} + 1$, $S_3(N_{S3}) = t_{suc}$
 Si $n3 > 0$, hacer
 Si $n3 < 5$, generar $R \sim \mathcal{N}(\mu_{31}, \sigma_{31})$
 en caso contrario, generar $R \sim \mathcal{N}(\mu_{32}, \sigma_{32})$
 Hacer $TSuc.t_{S3} = t_{suc} + R$
 Hacer $t = t_{suc}$
 Salir

Como en el sistema $G/G/1$, si el programa principal es ejecutado una única vez, se obtendrá una estimación bastante pobre de las cantidades demandadas. Por ello, será necesario replicar o repetir la ejecución del mismo obteniendo distintas estimaciones, las cuales se analizan mediante las técnicas de análisis de los resultados.

5.6. SSD DE UN MODELO DE INVENTARIO PROBABILÍSTICO

Un negocio o industria por lo general mantiene un inventario de sus productos para asegurar una operación ininterrumpida. Los mayoristas, los fabricantes e incluso los bancos de sangre, por lo general, almacenan bienes o artículos. ¿Cómo se decide en una instalación de este tipo su *política de inventario*, es decir, cuándo y cómo se reabastece? En una empresa pequeña, el administrador puede llevar un recuento de su inventario y tomar estas decisiones. Sin embargo, esto puede no ser factible en empresas medianas o grandes, en las que la aplicación de la *gestión del inventario* (Tersine, 1982) ha llevado al ahorro de grandes sumas de dinero.

Para ello se suele formular un modelo matemático que describe el comportamiento del sistema de inventarios; deriva una política óptima de inventarios con respecto a ese modelo, y, con frecuencia, utiliza un ordenador para mantener un registro de los niveles de inventario y señalar cuándo y cuánto conviene reabastecer. Sin embargo, para el modelo de inventario probabilístico que estudiamos en esta sección no se han desarrollado procedimientos analíticos para obtener sus principales medidas de interés, al no ajustarse a ninguno de los modelos analizados por la teoría de inventarios, por lo que será necesaria la simulación del comportamiento del sistema para poder obtener estimaciones de las cantidades de interés.

Consideramos un almacén de un determinado producto cuyo precio de venta al público es de r euros por unidad. La llegada de clientes al almacén se distribuye según un proceso de Poisson de parámetro λ . La cantidad de producto demandada por cada cliente sigue una distribución genérica G . Para satisfacer la demanda, el dueño del almacén mantiene un inventario del producto, de forma que cuando el nivel es bajo solicita al distribuidor más unidades. La *política de pedidos* al distribuidor es *periódica*: cuando el nivel del inventario es menor que una determinada cantidad p y no hay ningún pedido pendiente, se solicitan unidades del producto para que el nivel del inventario llegue a P unidades; si en el momento de realizar el pedido el nivel del inventario es x , entonces solicitamos $P - x$ unidades. El coste por unidad incluida en el pedido depende de la cantidad solicitada y según una función $c(y)$. Un pedido tarda L unidades de tiempo en ser servido (lapso de reabastecimiento), pagándose en ese momento. Adicionalmente, el dueño del almacén debe pagar una cantidad h por unidad de producto y unidad de tiempo, como coste de almacenamiento (alquiler del local, refrigeración...). En caso de que al llegar un cliente solicite una cantidad mayor que la que hay en

inventario, se le sirve lo que queda, perdiendo la venta restante.

En la Figura 5.9 se muestra un ejemplo de la evolución en el tiempo del nivel del inventario para las características identificadas. El empresario desea simular el comportamiento del almacén durante un periodo de tiempo T para estimar el beneficio esperado, la proporción de clientes cuya demanda se satisface completamente y el porcentaje de tiempo que el nivel del inventario permanece a cero. Para ello, supondremos que el nivel del inventario inicial es de P_0 unidades. El empresario utilizará esta información para redefinir su política de gestión del inventario.

En esta ocasión, los posibles sucesos que se pueden producir son dos, la llegada de un nuevo cliente al sistema y la llegada de un pedido del proveedor. Utilizaremos la estructura de datos $TSue\{t_C, t_P\}$, que nos permitirá saber en todo momento si hay algún suceso pendiente y, si es así, en qué instante se producirá, t_C para la llegada de clientes y t_P para la llegada de pedidos al proveedor.

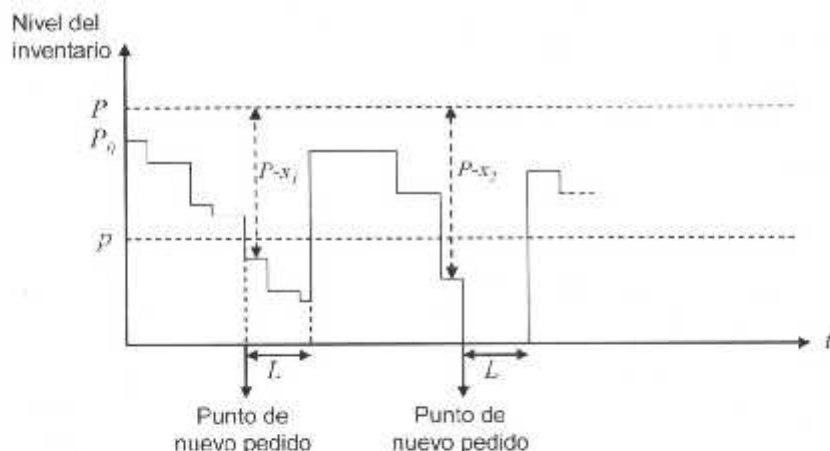


Figura 5.9: Evolución en el tiempo del modelo de inventario probabilístico

Otras variables que serán también necesarias son:

- x = número de unidades del producto en el inventario (variable de estado del sistema).
- y = número de unidades incluidas en un pedido pendiente. Si esta variable es cero, entonces no hay ningún pedido pendiente (variable de estado).
- t = tiempo transcurrido de simulación.

- var_nivel0 = variable en la que se guardarán los instantes en los que el nivel del inventario pasa a ser cero, y que nos servirá posteriormente para saber cuánto tiempo se ha mantenido vacío.

Las variables contador, que usaremos para tomar estadísticas y responder a la preguntas sobre el comportamiento del sistema, son:

- $C \equiv$ coste total por pedidos hasta el instante t .
- $H \equiv$ coste total por almacenamiento hasta el instante t .
- $R \equiv$ beneficios por ventas a clientes hasta el instante t .
- $N_{C+} \equiv$ número de clientes a los que hemos servido satisfactoriamente hasta el instante t .
- $N_{C-} \equiv$ número de clientes a los que no hemos podido satisfacer completamente su demanda hasta el instante t .
- $t_0 \equiv$ tiempo en el que el nivel del inventario está a cero.

Finalmente, las variables de salida serán:

- $beneficio \equiv$ beneficio esperado.
- $\%client_satisf$ = proporción de clientes cuya demanda se satisface completamente.
- $\%invent_0 \equiv$ porcentaje de tiempo que el nivel del inventario permanece a cero.

En el programa principal de simulación, análogo al de los modelos de colas analizados, el primer suceso que se produce es la llegada de un cliente al sistema (suponemos que $P_0 > p$), llamándose a la rutina que trata la ocurrencia de dicho suceso (Llegada_cliente). Dentro del bucle se llama a las rutinas que tratan los sucesos siguiendo el orden en que ocurren y, finalmente, calculamos las medidas de comportamiento del sistema en las que estábamos interesados a partir de las variables contador.

Hacer $x = P_0$, $y = N_{C-} = N_{C+} = C = R = H = 0$

Hacer $TSuc.t_C = TSuc.t_P = M$, $t = t_{suc} = t_0 = var_nivel0 = 0$

Generar $Z \sim \text{Exp}(\lambda)$

Si $Z > T$, hacer $\text{beneficio} = \%client_satisf = \%invent_0 = 0$, Salir
 en caso contrario, hacer
 Ejecutar **Llegada_cliente**(Z)
 Mientras $TSuc.t_P \neq M$ ó $TSuc.t_C \neq M$,
 Si $TSuc.t_C < TSuc.t_P$,
 Hacer $t_{suc} = TSuc.t_C$, $TSuc.t_C = M$
 Ejecutar **Llegada_cliente**(t_{suc})
 Si $TSuc.t_P < TSuc.t_C$,
 Hacer $t_{suc} = TSuc.t_P$, $TSuc.t_P = M$
 Ejecutar **Llegada_pedido**(t_{suc})
 Hacer $\text{beneficio} = R - C - H$, $\%invent_0 = t_0/t$
 Hacer $\%client_satisf = N_{C+}/(N_{C+} + N_{C-})$
 Salir

En la rutina en la que se trata la llegada de un cliente al sistema debemos en primer lugar actualizar los costes de almacenamiento hasta este instante, dependiendo del nivel del inventario (x) desde el instante en que ocurrió el suceso anterior, que se encuentra almacenado en la variable t hasta este momento. A continuación, actualizamos el tiempo transcurrido de simulación al instante actual, generamos la demanda asociada al cliente que acaba de entrar al sistema y comprobamos si podemos satisfacerla. Si el nivel actual del inventario es mayor o igual que la demanda del cliente realizamos la venta completa, para lo cual actualizamos la variable de ventas, disminuimos el nivel del inventario en la cantidad vendida, que es la demandada, y aumentamos el número de clientes cuya demanda se ha satisfecho íntegramente. Si el nivel del inventario es menor que la demanda del cliente, ésta no se podrá satisfacer completamente, en cuyo caso, aumentamos las ventas (se venden las unidades que quedaban de inventario), se actualiza el nivel del inventario a 0, se aumenta el número de clientes cuya demanda no se ha satisfecho íntegramente y guardamos en var_nivel0 el instante de tiempo actual, que nos servirá en un futuro para saber cuánto tiempo ha estado el nivel del inventario a 0.

Comprobamos si debemos realizar un nuevo pedido, viendo si el nivel del inventario en ese momento es menor que p . Si es así y, además, no hay ningún pedido pendiente, solicitamos la cantidad $P - x$ y guardamos en la lista de sucesos el instante en que llegará dicho pedido, que es el actual más el lapso de reabastecimiento. Finalmente, generamos el tiempo que transcurre hasta la llegada del siguiente cliente y si el instante en el que esta llegada tiene lugar no está fuera del horizonte de planificación guardamos en la lista de sucesos el instante en el que se producirá.

Llegada_cliente (t_{suc})

Hacer $H = H + (t_{suc} - t) \times h \times x$, $t = t_{suc}$

Generar $demanda \sim G$

Si $demanda < x$,

hacer $R = R + demanda \times r$, $x = x - demanda$, $N_{C-} = N_{C+} + 1$

en caso contrario,

hacer $R+ = R + (x \times r)$, $x = 0$, $N_{C-} = N_{C-} + 1$, $var_nivel0 = t$

Si $x \leq p$ e $y = 0$, hacer $y = P - x$, $TSuc.tp = t + L$

Generar $Y \sim Exp(\lambda)$

Si $t + Y < T$, hacer $TSuc.t_C = t + Y$

Salir

La rutina correspondiente a la llegada de un pedido al sistema actualiza los costes de almacenamiento hasta este instante. A continuación, actualizamos el tiempo transcurrido de simulación al instante actual, acumulamos el pago al distribuidor (que depende del número y de unidades provistas), aumentamos el nivel del inventario en las unidades del pedido, ponemos a cero el número de unidades pendientes de llegar y calculamos el tiempo que el nivel del inventario ha sido 0, si es que ha sido así, tomando la diferencia entre el instante actual y var_nivel0 .

Llegada_pedido (t_{suc})

Hacer $H = H + (t_{suc} - t) \times h \times x$

Hacer $t = t_{suc}$, $C = C + c(y)$

Hacer $x = x + y$, $y = 0$

Si $var_aux > 0$, hacer $t_0 = t_0 + (t - var_nivel0)$, $var_nivel0 = 0$

Salir

* De nuevo, será necesario replicar la ejecución del mismo obteniendo distintas estimaciones de las medidas de interés, las cuales serán analizadas mediante las técnicas introducidas en el Capítulo 8, de análisis de los resultados.

5.7. SSD PARALELA Y DISTRIBUIDA

La simulación paralela y distribuida de sucesos discretos se refiere a la ejecución de la SSD en una *ordenador paralelo o entorno distribuido*. La simulación secuencial de sistemas grandes y/o complejos es una actividad lenta, incluso cuando se utilizan los procesadores más avanzados. La utilización de múltiples procesadores permite mejorar la velocidad de la simulación explotando el posible paralelismo

inherente al sistema, ya que la mayoría de los sistemas simulados consisten en muchos componentes operando en paralelo.

Las dos ideas más utilizadas en paralelización en simulación son la réplica y la descomposición funcional. La *réplica* es la forma más obvia de explotar el paralelismo (Fujimoto, 1990). En muchas ocasiones, el estudio de simulación requiere múltiples ejecuciones del simulador con valores distintos para los parámetros de entrada. Se puede usar una computadora paralelo para ejecutar, simultáneamente, una simulación en cada procesador. El uso que se hace de los recursos, puede resultar excelente, aunque esta técnica no siempre es posible o conveniente. Por ejemplo, puede que no sea posible ejecutar simultáneamente varias simulaciones porque los parámetros de una utilicen los resultados de la anterior, situación habitual si el objetivo es encontrar los valores óptimos para un conjunto de parámetros.

La *descomposición funcional* trata de utilizar diferentes unidades funcionales para realizar simultáneamente las tareas necesarias en un simulador: generación de números aleatorios, gestión del reloj de simulación, entrada/salida, ejecución de los sucesos, representación gráfica, etc. El grado de paralelismo que se explota no es muy elevado, dado el escaso número de tareas identificables y la necesidad de coordinación entre ellas.

La *SSD distribuida* (Misra, 1986) se basa en la idea de *descomposición del modelo*. Se trata de descomponer el modelo a simular en un conjunto de partes o submodelos, asignando la simulación de cada una de esas partes a un *proceso lógico* (PL). Esos procesos lógicos se ejecutan en los procesadores disponibles. La descripción del modelo se distribuye entre los diferentes procesadores, por lo que el consumo de memoria en cada uno de ellos es razonablemente reducido.

Recordemos que las distintas componentes que forman un sistema real pueden interaccionar entre ellas, por lo que no es suficiente que cada submodelo se simule de forma concurrente con los demás, ya que habrá fuertes relaciones causa-efecto entre los sucesos que afectan a cada componente, debiendo recoger el simulador paralelo esta característica. Dicho de otra forma, los PLs que componen el simulador deben interaccionar para reflejar no sólo el progreso del submodelo que simulan, sino también la evolución del modelo global. Esto requiere que los PLs se coordinen entre ellos, de manera que las relaciones causales entre los sucesos que afectan a cada submodelo se respeten. Esta coordinación puede realizarse de diferentes maneras. La más restrictiva es hacer que los PLs trabajen de forma sincrónica, es decir, compartiendo la misma visión del transcurso del tiempo simulado. A efectos prácticos, esto significa que sólo se simulan en paralelo aquellos sucesos que en el sistema real ocurrirían simultáneamente. Esta técnica puede

dar buenos resultados si la densidad de sucesos es elevada.

En los *métodos de coordinación asíncronos* (Lin y Fishwick, 1995) los PLs interactúan entre sí intercambiando mensajes que representan sucesos del sistema. Cada PL tiene su propio reloj y decide de forma autónoma qué hacer con los sucesos que afectan a su parte del sistema, aunque debe hacerlo sin introducir errores causales, debidos a la diferencia de velocidad entre los procesadores en ejecución. Existen dos grandes familias de algoritmos asíncronos: los *conservadores* (Bryant, 1977; Chandy y Misra, 1979), que evitan la aparición de errores causales haciendo que un PL solamente avance en la simulación cuando esté seguro de que no supone ningún riesgo (mientras tanto, permanece bloqueado); y los *optimistas*, que no evitan errores de causalidad sino que los detectan y recuperan. En contraste con los mecanismos conservadores, no necesitan determinar cuándo es seguro proceder, sino que detectan un error e invocan un procedimiento de recuperación. El mecanismo *Time Warp* (Jefferson, 1985) es el protocolo optimista más conocido.

Uno de los problemas que plantean los sistemas de procesamiento paralelo disponibles en la actualidad es la enorme diversidad que existe tanto desde un punto de vista arquitectónico como en lo que se refiere a modelos de programación paralela. Desde el punto de vista de la programación de aplicaciones paralelas, se deben tener en cuenta aspectos como el modelo de paralelismo, el paradigma de comunicación (memoria compartida o paso de mensajes), la semántica de las operaciones de envío y recepción de mensajes o la identificación del interlocutor.

Al final de la próxima sección se presentarán algunos ejemplos de entornos para la simulación paralela y distribuida de sucesos discretos.

5.8. SOFTWARE DE SSD

Hemos visto que la descripción de un modelo SSD es sencilla. Sin embargo, la implementación de modelos complejos con varios tipos de sucesos y procesos y numerosas interacciones puede resultar complicada. Así, aunque cualquier simulación puede realizarse en lenguajes científicos de programación, como C, se han desarrollado numerosos productos de software específicos para SSD (dada su enorme importancia en aplicaciones industriales y de servicios).

Los *lenguajes de simulación* son programas de software versátiles que pueden utilizarse para distintas aplicaciones de modelización. En cierto sentido son comparables a lenguajes generales como C, Pascal... pero cuentan, además, con capacidades y herramientas específicas que facilitan la implementación de modelos de simulación. Esencialmente, tales capacidades evitan al analista crear herra-

mientas y procedimientos que aparecen en casi todas las aplicaciones de simulación, entre las que están:

- Un marco general para la creación y descripción de un modelo en términos de procesos, sucesos, entidades, atributos, recursos e interacciones entre componentes del modelo.
- La inclusión de un mecanismo de reloj de simulación que controla el avance del tiempo de simulación.
- La inclusión de métodos para secuenciar la ocurrencia de sucesos.
- Generadores de números aleatorios y de las principales variables aleatorias.
- Herramientas para la recolección y el análisis de estadísticas relativas al uso de recursos y otras entidades.
- Herramientas para la descripción gráfica del modelo y de la simulación.
- Herramientas para la validación y verificación del modelo.

Por otro lado, los lenguajes de simulación presentan como inconvenientes que suelen ser caros, no están disponibles para todos los sistemas operativos, requieren mayor tiempo de ejecución que los lenguajes generales.

Los lenguajes de simulación discreta disponibles entran en dos amplias categorías, en función de la estrategia de reloj de simulación empleada:

- **Programación de sucesos.** El usuario detalla las acciones asociadas a la ocurrencia de cada suceso. La contribución principal del lenguaje es automatizar el proceso de muestreo a partir de las distribuciones, el almacenamiento cronológico y recuperación de sucesos, así como la recopilación de estadísticas del modelo. *GASP* (Pritsker, 1974; Rinvall y Collier, 1982), *SLAM* (Pritsker y O'Reilly, 1999), *SIMAN* (Pegden *et al.* 1995) y *SIMSCRIPT* (Fayek, 1988) implementan este tipo de estrategia, aunque han evolucionado para permitir capacidades de orientación a procesos.
- **Orientados a procesos.** Usan bloques o nodos que se pueden unir para formar una red que describa los movimientos, transacciones o entidades en el sistema. Por ejemplo, los tres nodos más prominentes en cualquier lenguaje de simulación de procesos son la *fente* de la cual se crean las transacciones, la *cola* donde pueden esperar si es necesario y las *instalaciones* donde se lleva a cabo el servicio.

Se conducen internamente por las mismas acciones que se usan en los lenguajes de programación de sucesos. La diferencia es que esas acciones están automatizadas para liberar al usuario de detalles tediosos de cálculo. Están basados en el concepto de entrada-salida del enfoque de *caja negra*, proporcionando una mayor simplicidad y facilidad de uso a costa de una menor flexibilidad.

GPSS (Solomon, 1983; Karian y Dudewicz, 1999) fue el primero que se creó (en 1960 por IBM), está específicamente diseñado para la simulación de sistemas de espera, y requiere que el usuario domine el *trabajo interno* de aproximadamente 80 bloques diferentes. Otros lenguajes que implementan esta estrategia son *SIMULA* (Pooley, 1987; Holmevik, 1994) (<http://www.edelweb.fr/Simula/>) y *SIMNET II* (Taha, 1991) (<http://web.ineg.uark.edu/simnet2/simnet.htm>).

Algunos lenguajes, como *MODSIM* (Belanger, 1993) (<http://www.caci.com>) o *SIMSCRIPT II.5* (Russell, 1999) (<http://www.simscrip.com>), permiten modelizar con ambas estrategias.

Los *simuladores* son paquetes de software de sencillo manejo que permiten desarrollar modelos para situaciones específicas. Obviamente su uso es más fácil y permiten un desarrollo rápido de modelos, pero su escasa generalidad limita su interés como herramientas generales. Destaquemos que algunos simuladores se han desarrollado a partir de lenguajes de simulación. Por ejemplo, a partir de *SIMSCRIPT II.5*, se han desarrollado *LANNET II.5*, para simulación de redes de área local; *NETWORK II.5*, para simulación de redes de comunicaciones; y *SIMFACTORY II.5*, para simulación de sistemas de manufacturación, Garrison (1991). También *GPSS* evolucionó a *GPSS/H* (Banks *et al.*, 2003) que incluía entrada y salida por archivo y por pantalla y éste a *SLX* (Henriksen, 2000).

A continuación, se describen brevemente y se proporcionan enlaces en la web de algunos de los simuladores basados en sucesos discretos más conocidos, clasificándolos en función de sus aplicaciones:

Modelización y simulación de recursos naturales: *JDEVS* (<http://spe.univ-corse.fr/filippiweb/appli/index.html>), para modelización y simulación de sistemas naturales complejos, permite simulación de sucesos discretos, está orientado a objetos y conectado a un sistema de información geográfica (Filippi y Bisgambiglia, 2004); y *SEMOLA* (<http://www.dpvta.uniud.it/~Danuso/docs/Semola/homep.htm>), desarrollado por la Universidad de Udine y usado con éxito para modelizar sistemas biológicos, ecológicos y agrícolas con diferentes escalas y

niveles de dificultad (Danuso, 2001 y Danuso *et al.* 2004).

Simulación de redes: *cnet* (<http://www.csse.uwa.edu.au/cnet>), simulador de redes de ordenadores; *GloMoSim* (<http://pcl.cs.ucla.edu/projects/gloMosim>), entorno de simulación basado en el lenguaje de simulación Parsec para sistemas de redes inalámbricas (Zeng *et al.*, 1998); *ns2* (<http://www.isi.edu/nsnam/ns>), proporciona ayuda sustancial para la simulación de protocolos de TCP, enrutamiento y difusión sobre redes locales y por satélite (Fall y Varadhan, 2007); y *OMNeT++* (<http://www.omnetpp.org>), simulador de sucesos discretos modular, orientado a objetos, usado para modelizar protocolos de comunicación, redes de computadoras, multiprocesadores y sistemas distribuidos.

Simulación de procesos industriales: *AutoMod* (http://www.brookssoftware.com/pages/245_automod_overview.cfm) combina la facilidad de uso de un simulador con la flexibilidad y potencia de un lenguaje de simulación, usado en sistemas de fabricación y montaje (Banks, 2004); *FlexSim* (2005) (<http://www.flexsim.com>), puede ayudar a determinar la capacidad de una planta, gestionar cuellos de botella, resolver problemas de inventario, optimizar ratios de producción y justificar el gasto de capitales; *MODSIM III* (<http://www.caci.com>), desarrollado a partir del lenguaje de simulación MODSIM, con aplicaciones militares, en aeronáutica y sistemas de comunicaciones (Marti, 1999); *SIMAS II* (<http://www.cimpack.ch/products/products.html>) dedicado a la simulación en instalaciones de producción industrial en masa; *SimEvents 1.2* (<http://www.mathworks.com/products/simevents>) extensión de Simulink con herramientas para modelizar y simular sistemas de sucesos discretos usando colas y servidores; *SIMPLE ++*, usado para modelizar aplicaciones en la industria de la aviación, química, farmacéutica y metalúrgica; y *Spar* (<http://www.clockwork-solutions.com/tech-SPAR-modeling.shtml>), usado en gestión de flotas de aviones, análisis de procesos en plantas de producción, sistemas de distribución de electricidad.

Aunque se haya señalado que los simuladores construyen modelos para situaciones específicas, mostrándose distintos ejemplos, también se han desarrollado simuladores de propósito general. Entre ellos podemos incluir a *GPSS/H* y *SLX*, a los que se ha hecho referencia anteriormente. Otros dos, muy conocidos y utilizados, que permiten componentes discretos y continuos simultáneamente, son *Extend Suite* (Image, 2006) (http://www.imagethatinc.com/prods_suite.html), que permite una descripción gráfica del modelo para facilitar la implementación mediante la importación de bloques de librerías, conectados para indicar el flujo de entidades a través del sistema y detallados mediante el uso de cajas

de diálogo (en el Capítulo 11 se muestra un ejemplo de simulación en el que se utiliza esta herramienta); y *Arena* (Kelton *et al.*, 2004) (<http://www.arenasimulation.com/>). También son simuladores de propósito general *SIMUL8* (2005) (<http://www.simul8.com/>) y *HyPerformix Workbench* (Hyperformix, 2006) (<http://www.hyperformix.com/products/workbench/>).

Finalmente, algunos ejemplos de entornos para la simulación paralela y distribuida de sucesos discretos son *POSE* (Parallel Object-oriented Simulation Environment) (<http://charm.cs.uiuc.edu/research/pose>), desarrollado por el departamento de Informática de la Universidad de Illinois para modelizar sistemas de sucesos discretos complejos y especialmente para nuevas estrategias de sincronización optimistas y mixtas (Wilmarth y Kale, 2004); *COMPOSE* (Conservative, Optimistic and Mixed Parallel Object-oriented Simulation Environment) (Wang *et al.*, 1998); y *PARSEC* (Parallel Simulation Environment for Complex Systems) (<http://pcl.cs.ucla.edu/projects/parsec>) desarrollado por el laboratorio de computación paralela de la Universidad de UCLA y que incluye diferentes algoritmos de sincronización (Bagrodia *et al.*, 1998).

5.9. CONCLUSIONES

En el Capítulo 1 introdujimos un esquema de cuatro pasos para describir los experimentos de simulación. La ejecución del paso tercero dependerá esencialmente del tipo de modelo de simulación que consideremos y, por tanto, de la aplicación correspondiente. En este capítulo hemos proporcionado las ideas básicas sobre modelización en relación con la simulación y estudiado un grupo muy importante de aplicaciones denominado simulación de sucesos discretos. En capítulos posteriores estudiamos otras aplicaciones, así como el análisis e interpretación de los resultados de una simulación.

Se han presentado los conceptos básicos de la simulación de sucesos discretos ilustrados con varios ejemplos, dos de ellos orientados a sistemas de espera y el otro sobre un modelo de inventario probabilístico. Alguna bibliografía de interés centrada en la SSD es la siguiente: Banks *et al.* (2000) está especialmente orientado a lectores con poca base en cálculo, probabilidad y estadística, cubriendo de forma básica la SSD, mientras que Banks (1998) está orientado a lectores interesados en los aspectos más prácticos y novedosos de la SSD. Hace un especial hincapié en la aplicación en campos tan diversos como son el transporte, la sanidad o las aplicaciones militares; citando también aquellas empresas especialmente dedicadas al desarrollo y venta de productos software relacionados con este ámbito. Por otro lado, Fishman (2001) y Banks *et al.* (2005) son dos libros más

recientes centrados en la SSD. También son muchos los libros sobre simulación en los que se incluyen ejemplos de simulación de sistemas de espera complejos, como Ross (2002), que además del modelo $G/G/1$ incluye sistemas con dos servidores en serie y en paralelo; o Karian y Dudewicz (1999), en el que se analizan los sistemas con disciplinas de colas con prioridades.

A continuación, se ha introducido la simulación de sucesos discretos paralela (réplica y descomposición funcional) y distribuida (descomposición del modelo a simular en un conjunto de partes o submodelos, asignando la simulación de cada una de esas partes a un *proceso lógico*) que permiten mejorar la velocidad de la simulación. Se puede obtener información más detallada sobre paralelización y distribución en SSD en Fujimoto (2000) y Fujimoto *et al.* (2003).

Finalmente, se ha descrito el software existente actualmente en el mercado sobre simulación, tanto lenguajes de simulación (orientados a eventos y a procesos) como simuladores. En la revista *OR/MS Today* se realiza bianualmente un estudio del software en el mercado para simulación. En el último (Swain, 2007) se describen las características principales de 48 de ellos, incluyendo las aplicaciones típicas de cada uno, los requisitos de sistema (sistema operativo y RAM), sus características con respecto a la construcción de modelos (construcción gráfica, reutilización de código, ajuste de distribuciones de entrada...), capacidad de animación y precios. En la web <http://www.lionhrtpub.com/orms/surveys/Simulation/Simulation.html> está disponible el estudio de forma gratuita, incluyéndose algunas herramientas adicionales con respecto a la edición impresa. También en <http://www.topology.org/soft/sim.html> puede encontrarse información sobre software de SSD.

5.10. EJERCICIOS

1. En un centro de cálculo se considera que el uso del ordenador es excesivo y se piensa en reducir el acceso al mismo.
 - a) Los tiempos de procesamiento (en segundos) de los trabajos en un centro de cálculo siguen la distribución

Tiempo	1	2	3	4	5	6
Frecuencia	0.15	0.15	0.15	0.25	0.25	0.05

Esbócese un generador para esta distribución.

- b) Las llegadas de trabajos son según un proceso de Poisson de parámetro λ . Esbócese un generador para muestrear de este proceso.

- c) Describese la simulación orientada a sucesos del funcionamiento del centro de cálculo.
2. Supóngase un sistema formado por dos servidores en serie: los clientes que terminan su servicio en el primero de ellos pasan al segundo y cuando terminan su servicio en éste, abandonan el sistema. La llegada de clientes al sistema sigue un proceso de Poisson de parámetro λ y los tiempos de servicio en los servidores se distribuyen exponencialmente con parámetros μ_1 y μ_2 , respectivamente. Supóngase también que la disciplina de cola (de tamaño ilimitado) es FIFO en ambos servidores.

El sistema permite la entrada de clientes en el mismo hasta el instante T , momento en el cual no se permiten más entradas, pero sí se sigue dando servicio a los clientes que están en el sistema hasta que todos ellos lo abandonan.

Simúlese el comportamiento del sistema para estimar el tiempo transcurrido desde el instante T hasta que el último cliente abandona el sistema y el tiempo medio que pasan los clientes en el sistema y en cada uno de los servidores.

3. A un centro de cálculo llegan trabajos con distribución entre llegadas uniforme en $[4, 6]$ minutos. Los trabajos se procesan FIFO. La distribución de los tiempos de procesamiento es:

Probabilidad	Tiempo de procesamiento (min)
0.4	1
0.3	2
0.2	5
0.1	10

Los tiempos hasta caídas del sistema siguen una distribución uniforme en $[33, 43]$ horas. Los tiempos esperados de reparación se muestran en la siguiente tabla:

Hora de caída	Tiempo reparación (horas)
7 AM-7 PM	$U(3, 7)$
7 PM-7 AM	$U(11, 17)$

Se desea simular 7 días de operación del sistema, comenzando a las 7 AM. Coméntese la utilidad de este tipo de simulación en el proceso de toma de decisiones en el centro de cálculo.

4. El siguiente modelo sencillo de *Reed y Frost* describe una epidemia. La enfermedad se propaga sólo a partir de individuos infectados (casos) a otros (susceptibles) que no sean inmunes, por medio de un contacto adecuado. Los individuos se inmunizan tras tener la enfermedad. Al comienzo de la epidemia, hay una población p_0 con c_0 casos y $s_0 = p_0 - c_0$ susceptibles. Cada individuo tiene una probabilidad fija p de tener un contacto adecuado con cualquier otro individuo en cada instante. Una vez adquirida la enfermedad, se tiene durante un periodo después del cual se permanece inmune. El tamaño de la epidemia es el número de casos en cada periodo. Se pide:
 - a) Constrúyase un programa que permita la simulación de este modelo.
 - b) Explórese el impacto de los parámetros del modelo.
 - c) Discútase cómo puede hacerse más realista el modelo.
5. A un supermercado llegan clientes en intervalos de 2 ± 1 minutos. Éstos emplean en promedio 1 minuto para la compra de cada artículo; el número de artículos adquiridos sigue una distribución $\mathcal{P}(12)$. Hay seis cajas estándar que admiten clientes con cualquier número de artículos, dos rápidas que admiten compras de hasta un máximo de 5 artículos y una caja especial para disminuidos físicos. El número medio de clientes que utilizan la caja especial es del 4%. Prescindiendo del tiempo en la cola, cada cliente que pasa por una caja estándar emplea 6 segundos por artículo comprado, a través de una rápida emplea 5 segundos y, cuando se trata de un disminuido, emplea un total de 6 minutos. Se pide:
 - a) Simúlese la operación de pago de los clientes del supermercado durante un periodo de 10 horas.
 - b) Experimentétese con el modelo construido introduciendo cambios en los valores de los parámetros.
 - c) Estúdiense cuáles serían las recomendaciones más adecuadas para un funcionamiento eficiente del supermercado.
6. Un sistema de computación está servido por cinco impresoras. En promedio, resulta necesario imprimir 30.3 trabajos por hora y cada impresora es capaz de imprimir 7.4 trabajos por hora. Los tiempos entre llegadas de los trabajos, así como los tiempos de servicio de las impresoras, se distribuyen exponencialmente. Se pide:

- a) Simúlese el sistema de impresión y determínese cuál sería el número adecuado de impresoras para tener un servicio razonable.
 - b) Estúdiase el efecto de reemplazar en el sistema una de las impresoras actuales por otra más rápida, que es capaz de imprimir en promedio 9.2 trabajos a la hora.
7. Un sistema recibe sacudidas que le producen un deterioro y que ocurren de acuerdo con una distribución de Poisson con tasa una por hora. Se supone que los deterioros siguen variables aleatorias independientes (que también son independientes de los instantes en que se producen las sacudidas) con función de densidad común $f(x) = xe^{-x}$, $x > 0$. Los deterioros disminuyen con el tiempo con tasa exponencial λ , es decir, una sacudida que produce un deterioro inicial x mantendrá un deterioro con valor $xe^{-\lambda t}$ en el instante t después de haber ocurrido la sacudida. Se supone, además, que los deterioros son aditivos, es decir, que si hasta un instante s ha habido dos sacudidas, una en t_1 y la otra en t_2 , con deterioros iniciales x_1 y x_2 , respectivamente, entonces el deterioro total es $\sum_{i=1}^2 x_i e^{-\lambda(s-t_i)}$. El fallo en el sistema se produce cuando el deterioro total excede algún valor fijo M . Suponiendo que el interés del estudio de simulación se refiere a la estimación del tiempo medio hasta el fallo.
- a) Defínanse las componentes y variables en este modelo.
 - b) Escríbase el programa que realice la simulación.
 - c) Asignando los valores $\lambda = 0.4$ y $M = 6$, con $k = 1000$, utilícese la salida para estimar el tiempo medio hasta que el sistema falla.
8. Un sistema necesita n máquinas trabajando para que sea operativo. Si alguna máquina se estropea hay disponibles máquinas de repuesto, de modo que cuando una falla se sustituye inmediatamente. La máquina estropeada se envía para su reparación a un taller donde trabaja un único operario, que arregla las máquinas de una en una. Una vez que una máquina queda reparada, sirve como repuesto del sistema. Se supone que los tiempos de reparación son variables aleatorias i.i.d. con función de distribución G . Además, la duración de una máquina es una variable aleatoria, con función de distribución F . Se dice que el sistema se estropea cuando falla una máquina y no existe otra de repuesto. Si inicialmente hay $n + s$ máquinas de las cuales n están en funcionamiento en el sistema y s como repuesto. Se pide:

- a) Mediante simulación del sistema, indíquese cómo aproximar $E(T)$, donde T es el instante en que el sistema se estropea.
- b) Estímese $E(T)$ en el caso particular de $n = 5$, $s = 3$, $F(x) = 1 - e^{-x}$ y $G(x) = 1 - e^{-3x}$.
9. Consideremos un almacén de un determinado producto cuyo precio de venta al público es de 2 euros la unidad. La llegada de clientes al almacén se distribuye según un proceso de Poisson de parámetro $\lambda = 3$ clientes por hora y la cantidad de productos demandados por cada uno de ellos tiene la siguiente distribución:

Demanda	1 unidad	2 unidades	3 unidades	4 unidades
Probabilidad	0.3	0.4	0.2	0.1

Para satisfacer la demanda de sus clientes, el dueño del almacén mantiene un inventario de productos, de forma que cuando el nivel del inventario es bajo solicita al distribuidor más unidades. La *política de pedidos* al distribuidor es por lo tanto *periódica*: cuando el nivel del inventario es menor que una determinada cantidad (30 para nuestro problema) y no hay actualmente ningún pedido pendiente (esperando a que nos llegue), se solicitan unidades del producto para que el nivel del inventario llegue a 100 unidades. Por lo tanto, si en el momento de realizar el pedido el nivel del inventario es x , entonces se solicitan $100 - x$ unidades.

Asociado a cada pedido que realizamos, existe un coste fijo (coste de preparación) de 10 euros, independientemente de las unidades demandadas. Adicionalmente, el coste por unidad incluida en el pedido depende de la cantidad solicitada, habiendo descuentos por cantidad, es decir, si el número de unidades demandadas es menor que 50 el precio es de 1 euro por unidad, mientras que si se piden más de 50 el precio desciende a 75 céntimos. El tiempo que tarda en ser servido el pedido por los proveedores (lapso de reabastecimiento), sigue una distribución normal (truncada) de media 10 días y desviación típica 0.2, pagándose en ese momento. Se ha llegado a un acuerdo con el proveedor de forma que, tomando como referencia la semana que tarda en media un pedido en ser servido, por cada hora de retraso en la entrega del mismo se realizará un descuento del 0.05 % del valor del pedido, encareciéndose en la misma cantidad en el caso de que el pedido llegue con adelanto.

El dueño del almacén debe pagar 0.1 euros por unidad del producto y

día, asociado al almacenamiento físico de los productos (alquiler del local, refrigeración...)

En el caso de que al llegar un cliente éste solicite una cantidad mayor que la que hay en inventario, se le sirve lo que queda, perdiendo la venta restante. Esta situación supondrá una pérdida de imagen del negocio, pudiendo hacer que se pierdan clientes en un futuro. Para paliarlo, se ha decidido hacer un descuento del 20 % en las unidades servidas (no alcanzan su demanda) a estos clientes.

Simúlese el comportamiento del almacén durante un periodo de tiempo de 5 meses para estimar el beneficio esperado, la proporción de clientes cuya demanda se satisface completamente y el porcentaje de tiempo que el nivel del inventario permanece a cero. Para ello, supóngase que el nivel del inventario inicial es de 70 unidades del producto.